

SMARTBRIDGE / SKILL WALL

Sample Project Documentation

EduGenie: Google Gemini Powered Learning Assistant

Date: 30 September 2026 | Team ID: SWTID-2026-3580

Project Description: EduGenie is a Generative AI learning assistant for students. It provides Q&A, simple concept explanation, quiz generation, long-passage summarization and structured learning recommendations.

Scenarios: student Q&A; simplified topic explanation; four-option MCQ quiz; long-passage summary; beginner-to-advanced learning path.

Technical Architecture

EduGenie uses HTML/CSS/JavaScript for the interface, FastAPI/Uvicorn for REST APIs, Google Gemini for Q&A, quiz, summary and learning-path generation, and LaMini-Flan-T5-783M locally for concept explanation. The current MVP has no persistent database.

Pre-requisites: Python 3.x; FastAPI/Uvicorn; HTML/CSS/JavaScript; Google AI Studio/Gemini API key; Git/GitHub; REST/JSON basics.

Project Workflow

Milestone 1: Model selection and architecture.

Milestone 2: Core functionalities development.

Milestone 3: Backend development and API routes.

Milestone 4: Frontend development and dynamic results.

Milestone 5: Local run and functional verification.

Milestone 6: Conclusion and future enhancements.

Milestone 1: Model Selection and Architecture

Activity 1.1: Configure a Gemini API key securely through environment variables.

Activity 1.2: Review the current Gemini model catalogue and choose a supported generation model.

Activity 1.3: Define frontend, FastAPI, Gemini and local-model interactions.

Activity 1.4: Create the Python virtual environment and install dependencies.

Activity 1.1: Secure API Configuration

Create `.env` in the project root and set `GEMINI_API_KEY`. Optionally set `GEMINI_MODEL`. Load variables with `python-dotenv`. Never commit a real key. If exposed, revoke/rotate it.

Example: `GEMINI_API_KEY=your_gemini_api_key_here`

Activity 1.2: Model Selection

Review project requirements, context limits, supported methods, latency and structured-output behaviour. Keep the chosen model configurable through `GEMINI_MODEL` so the code does not depend on a retired model name.

Activity 1.3: Define Architecture

Frontend: templates/index.html and static/style.css with browser JavaScript.

Backend: main.py FastAPI application with validation and routing.

AI: gemini_client.py plus qna.py, quiz_module.py, summary_module.py and learning_path.py.

Local AI: explanation_module.py using LaMini-Flan-T5-783M.

Activity 1.4: Development Environment

Install Python 3.x, create and activate .venv, then run: `python -m pip install -r requirements.txt`. Configure .env locally. Keep secrets outside source control. Repository contains application modules, templates, static assets and tests.

Milestone 2: Core Functionalities Development

Activity 2.1: Q&A answers student questions; Explain simplifies concepts; Quiz generates MCQs with four options; Summary condenses long passages; Learning Path provides beginner, intermediate and advanced stages with resources.

Activity 2.2: FastAPI routes validate input, invoke modules and return JSON.

Milestone 3: Backend Development

main.py provides GET /, GET /health, POST /qa, POST /explain, POST /quiz, POST /summarize and POST /learn/recommendations. Backward-compatible /api routes are also retained. Each endpoint validates input and returns structured JSON.

Backend Request Processing

Student browser -> FastAPI endpoint -> input validation -> task module -> Gemini or LaMini model -> result -> JSON response -> browser.

Validation rejects blank or malformed input. Runtime model/API errors are separated from client validation errors.

Prompt Engineering and Reusable AI Client

`gemini_client.py` centralizes API configuration and model selection. Task prompts remain in independent modules so Q&A, quiz, summary and learning-path behaviour can be maintained separately. The local explanation model is isolated from Gemini calls.

AI Response Handling

Quiz output is parsed as structured JSON and checked for required fields. Text-generation results are cleaned before display. Empty model responses become runtime errors. API credentials are never returned to the browser.

Quiz structure: questions -> question, options[4], answer.

Milestone 4: Frontend Development

Activity 4.1: Build a responsive HTML/CSS/JavaScript learning workspace with task controls for Ask a Question, Explain a Topic, Summarize, Generate a Quiz and Learning Recommendations. Use clear inputs, result panels and readable contrast.

Activity 4.2: Dynamic Content Rendering

Submit requests asynchronously. Show loading state. Render text safely. Render quiz questions/options interactively. Display learning recommendations as beginner, intermediate and advanced sections. Restore controls after success or failure. Keep the API key server-side.

Milestone 5: Local Deployment

Activity 5.1: Activate the virtual environment, install requirements, create .env and configure GEMINI_API_KEY. Verify that GEMINI_MODEL is supported by the configured API. Start the application with Uvicorn.

Activity 5.2: Local Testing and Verification

Run: `uvicorn main:app --reload`

Open <http://127.0.0.1:8000>. Test Q&A, explanation, quiz, summary and learning recommendations. Call `/health`. Check empty-input validation and quiz answer rendering.

Activity 5.3: Deployment

The current submission is intended for local execution. A future public deployment should use production ASGI configuration, HTTPS and host-level environment secrets. Do not put `GEMINI_API_KEY` into source code or screenshots.

Deployment and Security Notes

Use environment variables for secrets. Use HTTPS for public deployments. Configure network/CORS access appropriately. The current MVP has no login system and no persistent learning-history database. A future deployment can add monitoring, authentication and storage.

Exploring the Website

Home / Learning Assistant Page: the single-page interface lets students choose a task, enter a question/topic/passage and view the result. Input areas are task-specific. Results show Q&A/explanation text, summaries, interactive quizzes and staged learning recommendations.

Conclusion

EduGenie is a Google Gemini powered learning assistant built with FastAPI, HTML, CSS and JavaScript, with a lightweight local LaMini-Flan-T5-783M explanation module. Future enhancements include voice, multilingual support, adaptive learning, persistent history, gamification, evaluation and scalable production deployment.

Repository: <https://github.com/aladiyan33/EduGenie>

Team ID: SWTID-2026-3580